

Rapport de quiz 2

Identification d'une Extension PHP



Stagiaire :

Yahya El Omari

1. Accès à l'Environnement

La première étape a consisté à accéder à l'environnement conteneurisé. L'accès au conteneur Docker a été effectué via une session interactive en ligne de commande :

```
mac@MBP-van-mac ~ % docker exec -it void-lab-php-ext bash
```

2. Exécution du Script et Observation

Une fois dans le répertoire de travail (/usr/src/myapp), le script principal a été exécuté :

```
root@b846d7c6b8fb:/usr/src/myapp# php app.php
=====
Hello You!
Find the the extension (.so) path and how it was loaded.
=====
```

L'inspection du code source avec la commande cat révèle la structure suivante :

```
root@b846d7c6b8fb:/usr/src/myapp# cat app.php
<?php

echo str_repeat("-", 10) . "\n";

\VoidLabs\canYouSeeMe();

echo str_repeat("-", 10) . "\n";
```

Le script appelle la fonction \VoidLabs\canYouSeeMe(). Cette fonction n'étant pas définie dans le fichier utilisateur, son exécution sans erreur fatale indique qu'elle est fournie par une extension chargée au niveau du moteur Zend.

3. Analyse des Extensions Chargées

L'inspection du répertoire système contenant les extensions compilées (/usr/local/lib/php/extensions/) liste les fichiers .so suivants :

- opcache.so
- sodium.so
- sample.so

Les modules opcache et sodium sont des composants standards. Le fichier sample.so est atypique et constitue l'extension personnalisée ajoutée à cet environnement.

4 4. Validation par l'API de Réflexion

L'utilisation de la réflexion interne de PHP permet d'inspecter les métadonnées de l'extension `sample` :

```
root@b846d7c6b8fb:/usr/src/myapp# php --re sample
Extension [ <persistent> extension #33 sample version 1.0 ] {

  - Functions {
    Function [ <internal:sample> function VoidLabs\canYouSeeMe ] {

      - Parameters [0] {
      }

      - Return [ void ]
    }
  }
}
```

La sortie confirme formellement que la fonction `VoidLabs\canYouSeeMe` est une fonction interne (`<internal:sample>`) exportée par le binaire `sample.so`.

5 5. Méthodologie et Intérêt des Extensions PHP

5.1 A. Procédure de création d'une extension PHP

D'après l'architecture détaillée dans la documentation technique de référence, le développement d'une extension nécessite une structure de fichiers précise et une séquence de compilation spécifique :

1. **Fichiers de configuration** : La définition des paramètres de compilation s'effectue via un fichier `config.m4` pour les systèmes de type Unix, ou `config.w32` pour les environnements Windows.
2. **Développement C** : Le code requiert des fichiers d'en-tête (`php_extension.h`) et des fichiers sources (`extension.c`). L'API Zend impose l'utilisation de macros telles que `PHP_FUNCTION` pour déclarer la logique, l'utilisation de `zend_parse_parameters` pour la réception des arguments de l'espace utilisateur, et la définition de la structure `zend_module_entry` pour l'enregistrement du module.
3. **Processus de compilation (3 étapes)** :
 - `phpize` : Exploite les en-têtes PHP et le fichier `config.m4` pour générer un script de configuration système.

- `./configure` : Exécute les vérifications spécifiques à l'environnement cible et génère un fichier `Makefile` adapté.
- `make` : Compile le code source en un binaire d'objet partagé final (`.so`).

5.2 B. Avantages techniques (Performance et Fonctionnalités)

Le développement sous forme d'extension C répond principalement à deux besoins :

- **Optimisation des performances** : Le code C étant compilé en langage machine, l'exécution d'algorithmes complexes est significativement plus rapide que l'équivalent interprété en PHP (ex : exécution d'un algorithme de simplification de polygones optimisée de 45 secondes à 0,37 seconde).
- **Capacités étendues** : Les extensions permettent d'interfacer PHP avec des bibliothèques système tierces, de conserver un état entre les requêtes (connexions persistantes), et d'interagir avec les structures internes du moteur.